



BasicIfStatements

🕒 Created time	July 13, 2026 5:09 AM
☰ Category	Conditionals
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 14, 2026 4:52 AM

```
bool isComplete = true;

if (isComplete)
{
    Console.WriteLine("The statement was true.");
    Console.WriteLine("This line works in true.");
}
else
{
    Console.WriteLine("The statement was false.");
    Console.WriteLine("This should also run");
}

//~~~~~

Console.Write("What is your first name: ");
string? firstName = Console.ReadLine();

string lastName;

if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello Mr. Corey.");
    lastName = "Corey";
}
else
{
    Console.WriteLine($"Helo {firstName}");
    lastName = "Smith";
}
```

```
Console.WriteLine(lastName);
Console.WriteLine("End of program.");
```

משפטי תנאי ולוגיקת הסתעפות

משפטי תנאי (**if statements**) הם יסודיים ובסיסיים מאוד לאופן שבו אנחנו עובדים עם שפת **C#**.
ה- **Conditional** מאפשר לנו ליצור לוגיקת הסתעפות (**Branching**).

אתחול משתנה בוליאני וערך ברירת המחדל

נתחיל עם טיפוס נתונים **bool**.

נאתחל אותו לערך ההתחלתי שלו, שהוא כברירת מחדל **false**.

לוגיקת הסתעפות (**Branching**) אומרת שהקוד לא ירוץ רק בקו ישר מלמעלה למטה, אלא מתפצל למסלולים שונים בהתאם להחלטות ולתנאים שהגדרנו.

בעבודה נכונה עם **C#**, הגדרת ערך ברירת מחדל כ-**false** למשתנה בוליאני מבטיחה שהקוד המותנה ירוץ אך ורק אם התנאי נבדק והתקיים במפורש והשתנה ל-**true**, מה ששומר על זרימת קוד ברורה ומונע שגיאות לוגיות.

~~~~~

## מבנה בסיסי של משפט if

קודם כל יש לנו את המילה **if**.

לאחר מכן יש לנו זוג סוגריים רגילים **()**, וכל מה שנמצא בתוך הסוגריים האלו יעבור הערכה ויבדק כדי לראות האם הוא **true**.

בלוק הקוד והסוגריים המסולסלים

```
if ()
{

}
```

יש לנו סוגריים מסולסלים פותחים **{** וסוגריים סוגרים **}**, וביניהם ימצא הקוד שירוצץ אך ורק אם מה שנמצא בתוך הסוגריים הרגילים **()** הוא **true**.

## שימוש במשתנה bool בתוך התנאי

יש לנו משתנה מסוג `bool`, והערך שלו יכול להיות `true` או `false`.  
אם נכתוב:

```
if (isComplete)
{
}
}
```

המבנה הזה יספק את דרישת משפט ה-`if` שלנו, מכיוון שהמערכת מסתכלת על המשתנה הבוליאני, והוא הרי מחזיק בערך של `true` או `false`.

## הוספת פקודות ביצוע והדפסה

אנחנו יכולים לכתוב:

```
if (isComplete)
{
    Console.WriteLine("The statement was true.");
}
```

ובתחתית הקוד, ממש בסוף, נכתוב את השורה:

```
Console.WriteLine("End of program.");
```

כך שזה יראה בצורה הבאה:

```
if (isComplete)
{
    Console.WriteLine("The statement was true.");
}
Console.WriteLine("End of program.");
```

## חשיבות הערך הבוליאני בזמן אמת

הנקודה הזו חשובה מאוד, כי במצב הנוכחי הערך של `isComplete` הוא `false`.  
המשמעות היא שהקוד שבתוך הסוגריים המסולסלים לא אמור לרוץ.  
אבל אם נגדיר את הערך שלו ל-`true`:

```
bool isComplete = true;
```

אז הקוד שלנו ירוץ, ואנחנו נראה בפלט את ההדפסה של השורה:

```
Console.WriteLine("The statement was true.");
```

```
bool isComplete = true;
if (isComplete)
{
    Console.WriteLine("The statement was true.");
}
Console.WriteLine("End of program.");
```

## סיכום אופן הפעולה של התנאי

לכן, בהתבסס על השאלה האם הביטוי או הערך הזה (`isComplete`) הוא `true` או `false`, המערכת תקבע האם נריץ את בלוק הקוד הבא או לא:

```
{
    Console.WriteLine("The statement was true.");
}
```

~~~~~

הצהרת `else` (בלוק "אחרת"):

משפט `else` פשוט אומר "אחרת", ויש לו סוגריים מסולסלים פותחים `{` וסוגריים מסולסלים סוגרים `}`. בתוך הבלוק הזה אנחנו יכולים לכתוב:

```
Console.WriteLine("The statement was false.");
```

וזאת רק כדי להראות לנו איזה חלק בקוד רץ כרגע.

דוגמת הקוד:

```
bool isComplete = true;

if (isComplete)
{
    Console.WriteLine("The statement was true.");
}
else
{
    Console.WriteLine("The statement was false.");
}
```

```
}  
  
Console.WriteLine("End of program.");
```

הסבר על זרימת הקוד:

עכשיו המצב הוא כזה: אם התנאי הוא **true**:

```
if (isComplete)
```

אז התוכנית תריץ את הבלוק הזה:

```
{  
    Console.WriteLine("The statement was true.");  
}
```

אחרת **else**, התוכנית תריץ את הבלוק הזה:

```
{  
    Console.WriteLine("The statement was false.");  
}
```

עקרון הריצה הבלעדית (אחד מהשניים):

המשמעות היא שלעולם לא יהיה לנו מצב שבו גם השורה הזו:

```
Console.WriteLine("The statement was true.");
```

וגם השורה הזו:

```
Console.WriteLine("The statement was false.");
```

ירוצו יחד באותה ריצה.

זה תמיד ירוץ או זה או זה.

שינוי הערך הבוליאני ל-**false**:

לכן, אם נשנה את המשתנה הבוליאני לערך **false**:

```
bool isComplete = false;
```

הקוד יציג עכשיו בפלט:

"The statement was false."

מכיוון שהתוכנית הריצה את בלוק ה-**else**.

זהו משפט **if/else** בסיסי.

~~~~~

לא היינו חייבים לכלול את בלוק ה-**else**.

אנחנו יכולים לכתוב בלוק **if** בלבד.

עם זאת, אנחנו לא יכולים לכתוב אך ורק בלוק **else** לבדו.

הסיבה לכך היא שחייב להיות לו קשר (יחס) למשהו קודם, כדי לדעת אם התנאי הוא **True** או **False**.

## המשמעות התכנותית ב-**#C**

### בלוק **if** כמשפט עצמאי

פקודת **if** היא יחידה עצמאית שבדקת תנאי ראשוני.

אם התנאי מחזיר **אמת**, הקוד שבתוך הבלוק ירוץ.

אם התנאי מחזיר **שקר**, התוכנית פשוט תדלג על הבלוק ותמשיך בשורה הבאה בקוד, ללא שום צורך בהוראה נוספת.

### בלוק **else** כמשפט תלוי

פקודת **else** אינה יכולה להתקיים בחלל ריק.

**המשמעות הלוגית שלה היא:** "בכל מקרה אחר שבו התנאי הקודם נכשל".

לכן, קומפילר ה-**#C** חייב לדעת למה ה-**else** מתייחס.

ללא בלוק **if** שקודם לו ומגדיר את תנאי הבסיס, הקומפילר לא ידע עבור איזה מצב **שקר** הוא אמור להריץ את הקוד, ותתקבל שגיאת קומפילציה.

~~~~~

אופרטור השוויון הכפול (**==**) הוא אופרטור השוואה שנמצא בתוך הסוגריים (**()**), והוא מחזיר לנו ערך **Boolean** **true** – או **false**, כמו בדוגמה:

```
if (firstName == "Tim")
```

האופרטור הזה משווה בין שני ערכים.

כלומר, הבדיקה בעצם שואלת: האם השם הפרטי שווה ל-"Tim".

כתוצאה מהבדיקה הזו, תוחזר לנו תשובה בוליאנית של **כן** או **לא** / **True** או **False**.

חשוב לזכור: הביטוי שנמצא בתוך הסוגריים **()** תמיד מחושב ומוערך לערך בוליאני:

```
(firstName == "Tim")
```

~~~~~

## רגישות לאותיות Case Sensitivity

מה קורה אם המשתמש כותב את השם בצורה כמו **tim**?

מדוע במצב שבו הערך היה **t** קטנה ו-**IM** גדולות, התוכנה לא הדפיסה **"Hello Mr. Corey"**?

הסיבה לכך היא תנאי ה-**if** שלנו:

```
if (firstName == "Tim")
```

הבדיקה הזו מחזירה **false**.

מדוע זה מחזיר **שקר**?

מכיוון שהשוואה **Case Sensitive**.

לכן, **T** גדולה ואחריה **im** באותיות קטנות זו הדרך היחידה שבה הערך יתאים בדיוק למחרוזת **"Tim"**.

## האם זה באמת מה שאנחנו רוצים?

לא, מכיוון שפתרון כזה אינו באמת חזק ועמיד מספיק להתמודדות עם שגיאות הקלדה של משתמשים.

## הפתרון: שימוש במתודה ToLower

קיימת מתודה של **string** שבה אנחנו יכולים להשתמש:

```
ToLower()
```

כאשר אנחנו כותבים את התנאי כך:

```
if (firstName.ToLower() == "tim")
```

המתודה ממירה את המחרוזת של השם הפרטי לאותיות קטנות לצורך ההשוואה.  
כעת, אין יותר חשיבות לאופן שבו המשתמש הזין את האותיות (גדולות או קטנות).

## דגש חשוב לגבי המשתנה המקורי והשימוש בפועל

**חשוב לציין:** השימוש במתודה הזו אינו משנה את הערך השמור במשתנה המקורי.  
אנחנו ככל הנראה נשתמש במתודה

```
ToLower()
```

לא מעט כאשר נדבר על מחרוזות ונבצע השוואת מחרוזות מהסוג הזה:

```
if (firstName.ToLower() == "tim")
```

~~~~~

תנאים מקוננים (Nested If Statements)

שימו לב, אנחנו יכולים ליצור תנאי if מקוננים (כלומר, תנאי if בתוך תנאי if אחר).

```
Console.Write("What is your first name: ");  
string? firstName = Console.ReadLine();  
if (firstName.ToLower() == "tim")  
{  
    Console.WriteLine("Hello Mr. Corey.");  
}  
else  
{  
    Console.WriteLine($"Hello {firstName}");  
}  
Console.WriteLine("End of program.");
```

הוספת תנאי if נוסף בתוך הבלוק

אנחנו יכולים להכניס בחלק הזה תנאי if נוסף:


```
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello Mr. Corey.");
}
```

גמישות בכתיבת הקוד ושימוש ב-Hard Code

זה תלוי אך ורק בנו אם אנחנו רוצים לעשות את זה, זה לגמרי אפשרי, ואנחנו יכולים להכניס לשם משתנים או עוד תנאי **if**.

לדוגמה, נייצר משתנה עם ערך הכתוב בקוד עצמו (**Hard Code**):

```
Console.WriteLine("What is your first name: ");
string? firstName = Console.ReadLine();
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello Mr. Corey.");
    string lastName = "Corey";
}
else
{
    Console.WriteLine($"Hello {firstName}");
}
Console.WriteLine(lastName);
Console.WriteLine("End of program.");
```

שגיאות קומפילציה וטווח הכרה (Scope)

שימו לב, ה-**Intellisense** לא מציג לנו את המשתנה, ואם ננסה להקליד אותו בכל זאת, נקבל הודעת שגיאה שאומרת:

```
"The name 'lastName' does not exist in the current context"
```

השם

```
'lastName'
```

אינו קיים בהקשר הנוכחי.

זה קורה בגלל שהמשתנה הזה נוצר בתוך ה-**Scope** הספציפי הזה:

```
{
    Console.WriteLine("Hello Mr. Corey.");
}
```

```
string lastName = "Corey";  
}
```

חוקי הגישה ל-Scope והצהרת משתנים

המשמעות היא שהמשתנה אינו יוצא החוצה מתוך ה-Scope הזה.

אנחנו יכולים להעביר נתונים לתוך Scope פנימי, אבל אי אפשר להוציא אותם החוצה ממנו.

אם אנחנו רוצים לשנות את הקוד כך שהמשתנה יהיה זמין לשימוש גם בחוץ, אנחנו צריכים להצהיר עליו בחלק העליון, ואז נוכל לכתוב:

```
lastName = "Corey";
```

המבנה התקין של הקוד

כך זה נראה בפועל לאחר השינוי:

```
Console.WriteLine("What is your first name: ");  
string? firstName = Console.ReadLine();  
string lastName;  
if (firstName.ToLower() == "tim")  
{  
    Console.WriteLine("Hello Mr. Corey.");  
    lastName = "Corey";  
}  
else  
{  
    Console.WriteLine($"Hello {firstName}");  
    lastName = "Smith";  
}  
Console.WriteLine(lastName);  
Console.WriteLine("End of program.");
```

גישה למשתנה מתוך ה-Parent Scope

אבל עכשיו, למרות ששינינו את הערך של המשתנה בתוך ה-Scope הפנימי, מכיוון שהוא נוצר מלכתחילה ב-Parent Scope, אנחנו יכולים לגשת אליו שוב ולהשתמש בו ב-Parent Scope ללא שום בעיה.